

Mount Kinematics and Pointing Errors in Tracking Simulations

GMT Technical Note — `trajectory-servos-mount`

Document ID: GMT-XXX-####	**Version**: Ver. X0
Date: 10/20/2026	**Status**: Draft
Author(s): Claude Opus 5.5 (Anthropic)	

Signatures

Author

Claude Opus 5.5, AI model (Anthropic)	Date

Approvers

Rod Conan	Date

Revision Log

Version	Date	Affected Sections	TNR #	Comments	Change Author
X0	10/20/2026	All	None		Claude Opus 5.5

Contents

1 Purpose **6**
1.1 Introduction . . . 6
1.2 Model overview . . . 7

2 Definitions, Acronyms, and Reference Documents **9**
2.1 Definitions . . . 9
2.2 Acronyms . . . 9
2.3 Applicable Documents . . . 9
2.4 Reference Documents . . . 9

3 Tracking trajectory **11**
3.1 Computation with astropy . . . 11
3.2 Mount setpoints and field rotation . . . 11

4 Mount drives feed-forward **12**
4.1 Why it is needed . . . 12
4.2 How it works . . . 12

5 Kinematic compensation **13**
5.1 Principle . . . 13
5.2 Kinematic gain from the FEM DC gain . . . 13
5.3 Compensation of the control feedback . . . 13
5.4 Consequence for the optics . . . 14

6 Pupil rotation **14**
6.1 Principle . . . 14
6.2 Implementation . . . 14

7 Telescope pointing errors **15**
7.1 Pointing error from the mount tracking error . . . 15
7.2 Field rotation error . . . 16
7.3 Optical model . . . 16

8 Verifications **17**
8.1 Trajectory and mount tracking . . . 17
8.2 Mount feed-forward . . . 17
8.3 Kinematic compensation . . . 20
8.4 Unit tests . . . 21
8.5 End-to-end verification of the image motion . . . 23

9 Computational performance **28**
9.1 Full trajectory . . . 28
9.2 Laptop and cloud GPUs . . . 28

List of Figures

1 ATP trajectory vs mount encoders (left) and tracking error (right), 300 s at 1 kHz. 19
2 Segment tip-tilt orbits. Left: FEM DC gain kinematics on the M1 and M2 rigid body motions only, 10–60 s. Right: kinematic compensation of all the FEM outputs, 30–300 s. . . . 21
3 M1 and M2 segment rigid body motions with kinematic compensation along the 300 s trajectory. . . . 22
4 Mount tracking error and image motion, 30 s on-axis run with GIR tracking. From top to bottom: tracking error, image centroid vs $-\boldsymbol{p}$, magnitudes of each term of Eq. (7.6), residual. . . . 25
5 Same as Figure 4 for a 30 s run with `GIR_TRACKING=0` and a source 0.25″ off-axis: the image rotates with the field. . . . 26
6 Same as Figure 4 for a 30 s run with GIR tracking, a source 0.25″ off-axis and the GIR tracking error exaggerated by 10^4 (`POINTING_ROTATION_GAIN=1e4`). . . . 27

List of Tables

1.1 Clients of the `trajectory-servos-mount` model. . . . 7
1.2 Signals of the model relevant to this note. Rates are given for the 1 kHz mount model. . . . 8
2.1 Definitions . . . 9
2.2 Acronyms . . . 10
2.3 Applicable Documents . . . 10
2.4 Referenced Documents . . . 10
6.1 Segment geometry used by `GirRotation`. . . . 15
8.1 Summary of the verifications. . . . 18
8.2 Mount axes cross-coupling $C_{k,l}$ of the feed-forward: averaged encoder angle of axis k (rows) per radian of feed-forward on axis l (columns), 2025 FEM. . . . 20
8.3 Image motion along y explained by the rotation term, 30 s run with GIR tracking, source 0.25″ off-axis and `PointingRotationError` exaggerated by 10^4 (1468 out of 1500 frames in the field). Residual rms [mas] without and with the rotation term, and fitted gain of the term. . . . 24
8.4 Image motion and residual rms [mas] of the 30 s on-axis run with GIR tracking (1472 out of 1500 frames in the field). . . . 24

9.1 Run time of the 300 s ATP trajectory with the `crseo` optical model, AWS g7e.4xlarge (RTX PRO 6000 Blackwell). 28
9.2 Run time of 30 s of the trajectory [s], two runs each. 29

1 Purpose

This note describes how the telescope motion along a sidereal tracking trajectory is modeled in the `trajectory-servos-mount` demonstration of the GMT integrated modeling framework (`dos-actors`, `demos/src/bin/trajectory-servos-mount`). It explains, and gives the equations implemented for:

- the generation of the tracking trajectory (Section 3),
- the mount drives feed-forward of the FEM rigid body springs (Section 4),
- the kinematic compensation of the rigid telescope motion in the FEM outputs (Section 5),
- the rotation of the pupil as seen by an instrument on the Gregorian Instrument Rotator (GIR) (Section 6),
- the telescope pointing and field rotation errors fed to the optical model (Section 7),

and it reports the verifications that were performed on each of them (Section 8).

1.1 Introduction

The integrated model couples the GMT structural finite element model (FEM), the mount control system, the M1 and M2 control systems (`GmtServoMechanisms`), a linear optical model (LOM) and, optionally, the `crseo` ray tracing optical model. The mount is driven along the 5 minutes ATP default tracking trajectory and the model is sampled at the mount control rate, 1 kHz for the default `MOUNT_FDR_1kHz` mount model. The FEM is the 2025 model `20251110_1617_zen_30_M1_202110_FSM_202305_Mount_202305_pier_202411_M1Fans` computed at 30° zenith angle.

The FEM is a linear model: a rotation of the mount by an angle θ appears in the FEM outputs as the linearized rigid displacement of every node of the telescope, superimposed on the structure deformations. Along a 300 s trajectory the azimuth axis rotates by 1.7°, which moves the M1 outer segments by about 0.25 m, far beyond the range of validity of a linear optical model. Three things are therefore required:

1. the mount drives must be able to hold the FEM away from its reference position, whose "rigid body" modes are in fact very soft springs (Section 4);
2. the rigid motion must be removed from the FEM outputs, leaving the deformations only (Section 5);

3. the part of the rigid motion that *does* matter optically, i.e. the difference between where the telescope points and where it should point, and the rotation of the instrument relative to the pupil and to the sky, must be reintroduced explicitly in the optical model (Sections 6 and 7).

1.2 Model overview

Table 1.1 lists the clients of the model involved in this note and Table 1.2 the signals exchanged between them.

Table 1.1: Clients of the `trajectory-servos-mount` model.

Client (source file)	Role
`Trajectory` (`main.rs`)	ATP trajectory, mount setpoints and field rotation
`SpringFeedForward` (`feedforward.rs`)	mount drive torques holding the FEM at the setpoint angles
`GmtServoMechanisms`	FEM with kinematic compensation, mount, M1 and M2 control systems
`LinearOpticalModel`	segment tip-tilt from the M1 and M2 rigid body motions
`GirRotation` (`gir_rotation.rs`)	M1 and M2 rigid body motions rotated by the GIR angle
`Pointing` (`pointing.rs`)	telescope pointing and rotation errors from the mount tracking error
`OpticalModel<Camera<1>>`	ray tracing optical model and imaging camera

Table 1.2: Signals of the model relevant to this note. Rates are given for the 1 kHz mount model.

Signal	Rate	Content
`MountSetPoint`	1 kHz	azimuth, elevation and GIR setpoints [rad]
`FieldRotation`	1 kHz	change of the parallactic angle [rad]
`MountTorquesFeedForward`	1 kHz	8 azimuth, 4 elevation and 4 GIR drive torques [N m]
`MountEncoders`	1 kHz	all the mount encoders, GIR encoders at indices 10–13
`AverageMountEncoders`	1 kHz	averaged azimuth, elevation and GIR encoders [rad]
`M1/M2RigidBodyMotions`	1 kHz / 50 Hz	7×6 segment rigid body motions in the segment local frames
`PointingError`	50 Hz	telescope optical axis relative to the source, (x, y) in the OSS [rad]
`PointingRotationError`	50 Hz	instrument rotation about the optical axis relative to the sky [rad]
`TipTilt`	50 Hz	wavefront tip-tilt, ordered $[y, x]$ [rad]
`Frame<Host>`	50 Hz	256×256 camera frame

2 Definitions, Acronyms, and Reference Documents

2.1 Definitions

Table 2.1: Definitions

Term	Definition
OSS	Optical Support Structure coordinate system: z along the optical axis, x along the elevation axis
Tracking error	mount encoders minus mount setpoint
Field rotation	change of the parallactic angle along the trajectory, followed by the GIR
Kinematic gain	static gain K from the averaged mount encoder angles to the FEM outputs
Absolute pointing	camera pointing fixed with respect to the optical axis, the image moving with the source direction

2.2 Acronyms

2.3 Applicable Documents

2.4 Reference Documents

Table 2.2: Acronyms

Acronym	Description
ATP	Acquisition, Tracking and Pointing (TBC)
CEO	CUDA Engined Optics
DC	Direct Current (static, zero frequency)
DFT	Discrete Fourier Transform
FEM	Finite Element Model
FOV	Field Of View
GPU	Graphics Processing Unit
GIR	Gregorian Instrument Rotator
GMT	Giant Magellan Telescope
GMTO	Giant Magellan Telescope Corporation
ICRS	International Celestial Reference System
LOM	Linear Optical Model
LST	Local Sidereal Time
OSS	Optical Support Structure
RBM	Rigid Body Motion
TETE	True Equator True Equinox

Table 2.3: Applicable Documents

Document Number	Title	Manage Link
TBD		

Table 2.4: Referenced Documents

Document Number	Title	Manage Link
GMT-REF-00362	GMT Project Acronyms and Glossary	https://bit.ly/3l4reUC
RD1	`trajectory-servos-mount` demo: README, sources and scripts (`demos/src/bin/`)	`dos-actors` repository
RD2	`gmt_dos-clients_fem` kinematic compensation (`kinematics.rs`)	`dos-actors` repository
RD3	`gmt_dos-clients_crseo` pointing errors (`optical_model.rs`)	`dos-actors` repository

3 Tracking trajectory

3.1 Computation with astropy

The trajectory `atp_trajectory.csv` is computed by `scripts/atp_trajectory.py` (`make trajectory`) with astropy for the ATP default pointing:

- site: Las Campanas, latitude $\phi = -29.049°$, longitude $-70.682°$, height 2514 m;
- epoch: $t_0 =$ 2018-01-01T04:00:00 UTC;
- initial pointing: altitude 45°, azimuth 0°, rotator 0°.

The sky target is fixed by converting the initial altitude and azimuth at t_0 to ICRS coordinates, without atmospheric refraction ($\alpha = 89.8175°$, $\delta = 15.9532°$). The target is then transformed back to the local horizontal frame (`AltAz`) at 301 epochs, $t_0 + k\,\mathrm{s}$, $k = 0, \ldots, 300$, giving the elevation and the azimuth, the latter unwrapped to avoid the $\pm 180°$ discontinuity.

The rotator angle is the change of the parallactic angle

$$q = \mathrm{atan2}(\sin H,\ \tan\phi \cos\delta - \sin\delta \cos H)\,, \tag{3.1}$$

where the hour angle H is the apparent local sidereal time minus the target right ascension in the TETE frame (equinox based apparent place, consistent with the apparent LST), and δ the TETE declination. q is unwrapped and the rotator angle is $q(t) - q(t_0)$.

The file holds 4 columns: time [s], azimuth, elevation and rotator [deg]. Over 300 s the azimuth changes by $-1.704°$ ($-6134''$, about $-20.4''$/s), the elevation by $-58.7''$ and the rotator by $-1.549°$ ($-5577''$, about $-18.6''$/s).

3.2 Mount setpoints and field rotation

The FEM is linearized about its reference configuration, so the mount setpoints are the offsets of the trajectory from its first sample. The `Trajectory` client reads the file, converts the angles to radians, subtracts the first sample, and linearly interpolates the 1 Hz samples to the mount sampling frequency f_s: at step n, with $i = \lfloor n/f_s \rfloor$ and $a = (n \bmod f_s)/f_s$,

$$\boldsymbol{s}(n) = \boldsymbol{s}_i + a\,(\boldsymbol{s}_{i+1} - \boldsymbol{s}_i), \tag{3.2}$$

where $\boldsymbol{s}_i = [\mathrm{az}_i, \mathrm{el}_i, q_i]$. It writes two outputs:

- `MountSetPoint`: $[\mathrm{az}, \mathrm{el}, q]$, or $[\mathrm{az}, \mathrm{el}, 0]$ when `GIR_TRACKING=0` (the GIR is at rest);
- `FieldRotation`: q, in both cases, since the field rotates whether or not the GIR follows it.

4 Mount drives feed-forward

4.1 Why it is needed

In the FEM, the azimuth, elevation and GIR "rigid body" modes are not free: they have small but non-zero eigen frequencies, between 0.07 Hz and 0.1 Hz. Each mount axis therefore behaves as a very soft spring, and holding it at an angle θ away from the FEM reference requires a static drive torque proportional to θ. In the mount controller this torque has to be built up by the integral action of the feedback loop. Along the trajectory the azimuth angle grows linearly, so does the required torque, and the azimuth drives reach their 800 kN m saturation at about 242 s: the mount then stops tracking. The spring is an artifact of the FEM, a real mount does not need a torque to hold an angle, so it is cancelled by a feed-forward rather than by retuning the controller.

4.2 How it works

`SpringFeedForward::from_fem` derives, from the FEM, the torque per drive and per radian of each axis. The static (DC) gain from the drive torques $\boldsymbol{\tau}$ (8 azimuth, 4 elevation and 4 GIR drives) to any FEM output $\boldsymbol{y}$ is computed from all the FEM modes,

$$\boldsymbol{y} = \Phi\,\Omega^{-2}\,B\,\boldsymbol{\tau} = G\,\boldsymbol{\tau}, \tag{4.1}$$

with B the inputs to modal forces matrix (`in2modes`) of the drive torques, Ω^2 the diagonal matrix of the squared modal angular frequencies ω_i^2 and Φ the modes to outputs matrix (`modes2out`). Applying Eq. (4.1) to the encoders of each axis and averaging over the encoders of the axis gives the gain $g_{k,j}$ from drive j to the averaged encoder angle of axis $k \in \{\mathrm{az}, \mathrm{el}, \mathrm{GIR}\}$. If the same torque τ_k is applied on the n_k drives $\mathcal{D}_k$ of axis k, the axis moves by $\theta_k = \tau_k \sum_{j\in\mathcal{D}_k} g_{k,j}$, so the torque per drive that holds the axis at the setpoint θ_k is

$$\tau_k = \kappa_k\,\theta_k, \qquad \kappa_k = \Big(\sum_{j\in\mathcal{D}_k} g_{k,j}\Big)^{-1}. \tag{4.2}$$

At each step the client reads `MountSetPoint` and writes `MountTorquesFeedForward`, the 16 torques $[\kappa_{\mathrm{az}}\theta_{\mathrm{az}}\times 8,\ \kappa_{\mathrm{el}}\theta_{\mathrm{el}}\times 4,\ \kappa_{\mathrm{GIR}}\theta_{\mathrm{GIR}}\times 4]$. The mount controller (`gmt_dos-clients_mount`) adds them to the drive torques *after* the drives saturation, so the saturation only applies to the feedback torques. The cross-coupling between axes is neglected: only the diagonal terms $g_{k,j}$, $j \in \mathcal{D}_k$, are used; the coupling is estimated in Section 8.2 and found negligible. The stiffness $[\kappa_{\mathrm{az}}, \kappa_{\mathrm{el}}, \kappa_{\mathrm{GIR}}]$ is printed at start-up; for the 2025 FEM it is $[4.17\times10^6,\ 1.08\times10^7,\ 1.00\times10^5]$ N m/rad per drive. `MOUNT_FEEDFORWARD=0` turns the feed-forward off.

5 Kinematic compensation

5.1 Principle

As the mount moves, every FEM output $\boldsymbol{y}$ (segment rigid body motions, hardpoint forces, M2 positioners, ...) includes the motion that follows the mount, i.e. the rigid telescope motion plus the static deflection of the structure that comes with it. The kinematic compensation removes from each output the part that is linearly related to the averaged mount encoder angles $\boldsymbol{\theta} = [\bar{\theta}_{\text{az}}, \bar{\theta}_{\text{el}}, \bar{\theta}_{\text{GIR}}]$:

$$\boldsymbol{y}_c = \boldsymbol{y} - K\,\boldsymbol{\theta}, \tag{5.1}$$

where K is the $n_y \times 3$ kinematic gain. The compensation is applied in the FEM client after each state update (`DiscreteModalSolver::update`), so that every consumer of the FEM outputs, including the M1 and M2 control systems, sees compensated outputs.

5.2 Kinematic gain from the FEM DC gain

K is derived from the same static gain as the feed-forward, Eq. (4.1), computed in `DiscreteStateSpace::kinematic_compensator`. Let H be the $n_y \times 16$ DC gain from the 16 drive torques to all the FEM outputs, and E the 3×16 DC gain from the drive torques to the averaged azimuth, elevation and GIR encoders (the row mean of H over the encoders of each axis). A static torque $\boldsymbol{\tau}$ produces $\boldsymbol{y} = H\boldsymbol{\tau}$ and $\boldsymbol{\theta} = E\boldsymbol{\tau}$. The kinematic gain is the least-squares map from $\boldsymbol{\theta}$ to $\boldsymbol{y}$ over the torques:

$$K = HE^{+}, \qquad E^{+} = E^{T}\left(EE^{T}\right)^{-1}. \tag{5.2}$$

Each column of K is the static response of the outputs per radian of one mount axis, as measured by its encoders. Because K is computed from all the FEM modes, it contains the exact linearized rigid motion of the FEM (with its actual geometry and axes) and the quasi-static flexure that goes with it, with no geometric model of the telescope.

The rows of K of the mount encoders are set to zero: the mount control system needs the uncompensated encoders. Other outputs can be left uncompensated (`kinematic_exclusion`), and a column of K can be replaced by a calibrated gain (`kinematic_calibration`); neither is used in the demo, where `GmtServoMechanisms::kinematic_compensation` compensates all the outputs but the mount encoders.

5.3 Compensation of the control feedback

The outputs fed back to the control systems must be compensated too. The M1 hardpoint forces, in particular, contain a kinematic component as the telescope rotates: the M1 actuators force loop would offload it and make the M1 segments drift with the mount azimuth (Section 8.3). Compensating all the FEM outputs removes the drift without closed-loop calibration.

5.4 Consequence for the optics

After compensation the M1 and M2 rigid body motions only contain the structure deformations relative to the mount encoders (Figure 3). The rigid telescope motion, including the tracking error, has been removed with the encoders. That is the desired behavior for the large trajectory motion, which the sky follows, but it also removes the tracking error, which does move the image. The tracking error is therefore reintroduced as a pointing error in the optical model (Section 7). `KINEMATIC_COMPENSATION=0` runs the FEM without compensation.

6 Pupil rotation

6.1 Principle

The FEM outputs are given in the OSS, which is fixed to the telescope tube. An instrument mounted on the GIR rotates with it about the optical axis by the GIR angle γ, and sees the pupil (the M1 and M2 segments) rotated with respect to its own frame. With the optical model feature (`gmt_dos-clients_crseo`), `GirRotation` (`gir_rotation.rs`) rotates the M1 and M2 segments as rigid bodies about the OSS z axis by γ, the mean of the 4 GIR encoders (`MountEncoders` indices 10–13, uncompensated), before ray tracing.

6.2 Implementation

Consider a segment with vertex $\boldsymbol{r}$ and local frame Q ($\boldsymbol{v}_{\mathrm{OSS}} = Q\,\boldsymbol{v}_{\mathrm{local}}$), deformed by the compensated rigid body motion $(\boldsymbol{t}_d, R_d)$ from the FEM, given in its local frame. The deformed segment vertex is at $\boldsymbol{r} + Q\boldsymbol{t}_d$ and its orientation is QR_d in the OSS. Rotating the deformed mirror by $R_z(\gamma)$ and expressing the result in the nominal local frame gives the rigid body motion fed to the optical model:

$$\boldsymbol{t} = Q^T\,(R_z(\gamma) - I)\,\boldsymbol{r} + M\,\boldsymbol{t}_d, \tag{6.1}$$

$$R = M R_d, \qquad M = Q^T R_z(\gamma)\,Q, \tag{6.2}$$

M being the mirror rotation in the segment local frame. The rotations are given as the (x, y, z) Euler angles of the ray tracing coordinate systems, $R = R_z(c)R_y(b)R_x(a)$, from which

$$b = -\arcsin R_{20}, \quad a = \operatorname{atan2}(R_{21}, R_{22}), \quad c = \operatorname{atan2}(R_{10}, R_{00}). \tag{6.3}$$

The deformation thus rotates with the mirror, as it does seen from an instrument on the GIR. A first implementation added the rotation of the undeformed mirror to the deformation, $\boldsymbol{t} = Q^T(R_z(\gamma) - I)\boldsymbol{r} + \boldsymbol{t}_d$ and the sum of the Euler angles of M and R_d: the deformation was left in the orientation of the telescope tube, an error of the order of γ times the deformation. Over the 30 s run of Section 8.5 ($\gamma \leq 0.15°$), the exact composition changes the wavefront

tip-tilt by 1.3×10^{-3} mas on average and at most 6×10^{-3} mas; at the end of the 300 s trajectory ($\gamma = 1.55°$) the error would have been about 2.7% of the deformation terms. The segment geometry (Table 6.1) follows the geotrans conventions: the 6 outer segments at a radial distance ρ with clocking $o_i = o_0 - 60°i$, the vertex height being the mirror height plus the conic sag $z(\rho) = \rho^2/\left(R(1+\sqrt{1-(1+k)\rho^2/R^2})\right)$, and the local frames $Q_i = R_z(o_i)R_x(\beta)$ for M1 and $Q_i = R_z(o_i)R_y(\pi)R_x(\beta)$ for M2. The central segment (S7) frame is the identity for M1 and $R_z(\pi)R_y(\pi)$ for M2, whose S7 frame is flipped with respect to the OSS.

Table 6.1: Segment geometry used by `GirRotation`.

	M1	M2
outer segment distance ρ [m]	8.71	1.08774
clocking o_0 [deg]	0	180
vertex height [m]	3.9	3.9 + 20.26247614
radius of curvature R [m]	36	−4.1639009
conic constant k	−0.9982857	−0.71692784
outer segment tilt β [deg]	13.601685	14.777498

The rotated rigid body motions are fed to the optical model at 50 Hz. Rotating the pupil rotates the diffraction pattern of the segmented aperture (and any segment piston/tip-tilt pattern) on the detector; it does not move the image centroid of a perfectly phased telescope, which is set by the source direction (Section 7).

7 Telescope pointing errors

7.1 Pointing error from the mount tracking error

The `Pointing` client (`pointing.rs`) reads `MountSetPoint`, `AverageMountEncoders` and `FieldRotation` at 1 kHz. The tracking errors are $\Delta\mathrm{az} = \bar{\theta}_{\mathrm{az}} - s_{\mathrm{az}}$ and $\Delta\mathrm{el} = \bar{\theta}_{\mathrm{el}} - s_{\mathrm{el}}$. In the OSS, with ζ the zenith angle, the azimuth axis is $(0, \sin\zeta, \cos\zeta)$ and the elevation axis is $(-1, 0, 0)$. The telescope rotation vector is

$$\boldsymbol{w} = \Delta\mathrm{az}\,(0, \sin\zeta, \cos\zeta) + \Delta\mathrm{el}\,(-1, 0, 0), \tag{7.1}$$

which moves the optical axis $\hat{\boldsymbol{z}}$ by $\boldsymbol{w} \times \hat{\boldsymbol{z}} = (w_y, -w_x, 0)$, i.e. the pointing error in the OSS is

$$\boldsymbol{p}_{\mathrm{OSS}} = (\Delta\mathrm{az}\cos(\mathrm{el}),\ \Delta\mathrm{el})\,. \tag{7.2}$$

The elevation is the telescope elevation, $\mathrm{el} = \mathrm{el}_0 + \bar{\theta}_{\mathrm{el}}$, with el_0 the initial elevation of the trajectory (45°, read from the first row of the trajectory file) and $\bar{\theta}_{\mathrm{el}}$ the averaged elevation encoder, the encoders being offsets from the initial pointing. Along the ATP trajectory the

elevation decreases by $58.7''$, changing cos(el) by 3×10^{-4} in relative terms. As the mirrors (Section 6), the pointing error is rotated by the GIR angle $\gamma = \bar{\theta}_{\mathrm{GIR}}$ into the instrument frame:

$$\boldsymbol{p} = R(\gamma)\,\boldsymbol{p}_{\mathrm{OSS}}, \qquad R(\gamma) = \begin{pmatrix} \cos\gamma & -\sin\gamma \\ \sin\gamma & \cos\gamma \end{pmatrix}. \tag{7.3}$$

$\boldsymbol{p}$ is the `PointingError` output.

7.2 Field rotation error

The GIR compensates the field rotation q (Section 3.2). Its tracking error relative to the sky,

$$\Delta\gamma = \bar{\theta}_{\mathrm{GIR}} - q, \tag{7.4}$$

is the `PointingRotationError` output. With the GIR tracking, $\Delta\gamma$ is the GIR tracking error; with `GIR_TRACKING=0`, the GIR is at rest and $\Delta\gamma = -q$: the field rotates on the instrument. `POINTING_ERROR=0` sets both outputs to zero, and `POINTING_ROTATION_GAIN` scales $\Delta\gamma$ for verification only (Section 8.5). Both outputs are decimated to 50 Hz, the rate of the optical model.

7.3 Optical model

The optical model images a J band source (1.215 µm) at an off-axis angle θ_0 (`OFF_AXIS`, default $0.25''$ along x) on a camera with a $\lambda/(5D)$ DFT sampling binned 2×2, i.e. a 3.93 mas pixel over a 256×256 frame ($\pm 0.5''$ FOV). The camera is in *absolute pointing* along the optical axis (`sensor_pointing`$(-\theta_0, 0)$): the image sits at the source direction seen from the instrument, θ_0 from the frame center, and moves when that direction changes. This requires `crseo-sys` 1.3.3 or later (CEO imaging fix, the x axis of the absolute pointing along the frame columns).

When the optical model reads `PointingError` or `PointingRotationError`, the source directions are set to the directions seen from the instrument (`Pointing::apply` in `gmt_dos-clients_crseo`): for a nominal source at $(x, y) = \zeta_s(\cos a_s, \sin a_s)$,

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = R(-\Delta\gamma) \begin{pmatrix} x \\ y \end{pmatrix} - \boldsymbol{p}, \tag{7.5}$$

the nominal directions being kept unchanged. The source is ray traced along (x', y'), so the image moves by $-\boldsymbol{p}$ and is rotated by $-\Delta\gamma$ about the optical axis. For the off-axis source, the rotation moves the image by $\simeq -\Delta\gamma\,\theta_0$ perpendicular to its field direction. The wavefront tip-tilt (`TipTilt`, ordered $[y, x]$) is computed relative to the source direction, and thus contains the structure deformations only. The expected image motion is

$$\delta\boldsymbol{c} = \left(R(-\Delta\gamma) - I\right) \begin{pmatrix} \theta_0 \\ 0 \end{pmatrix} - \boldsymbol{p} + \boldsymbol{t}_{\mathrm{TT}}, \tag{7.6}$$

with $\boldsymbol{t}_{\mathrm{TT}}$ the wavefront tip-tilt. Since the kinematic compensation has removed the rigid telescope motion measured by the encoders, Eq. (7.5) is the only path by which the mount tracking error reaches the optics.

Magnitudes along the trajectory: the GIR tracking error peaks at about 6.9″ during the mount start-up transient, i.e. $\Delta\gamma\,\theta_0 \lesssim 0.008\,\mathrm{mas}$ at 0.25″. Without GIR tracking the field rotates by 0.155° in 30 s and 1.55° in 300 s, moving the off-axis image by 0.68 mas and 6.8 mas, respectively.

8 Verifications

Table 8.1 summarizes the verifications; they are detailed below. Unit tests were re-run on 10/20/2026 for this note; the integrated model results are from the runs whose data and figures are stored in the demo `data/` and `figures/` folders.

8.1 Trajectory and mount tracking

Running `scripts/atp_trajectory.py` again produced a file identical to the committed `atp_trajectory.csv`; the script reports the target ICRS coordinates, the azimuth, elevation and rotator ranges of Section 3.1, and an initial parallactic angle of −180°, as expected for a target on the meridian ($H = 0$) north of a southern site.

Figure 1 compares the 1 Hz trajectory to the averaged mount encoders over the full 300 s at 1 kHz. The encoders follow the interpolated setpoints; the tracking error is a start-up transient of the first 2 s (up to 1.4″ in azimuth, 0.02″ in elevation and 6.9″ for the GIR) after which it is negligible, with a small GIR ripple.

8.2 Mount feed-forward

Without feed-forward (`MOUNT_FEEDFORWARD=0`) the azimuth drives saturate at about 242 s and the mount stops following the trajectory. With the feed-forward of Eq. (4.2) the full 300 s trajectory runs without saturation and with the tracking error of Figure 1: the integrator of the mount controller no longer has to build up the spring torque.

The cross-coupling neglected by the feed-forward is estimated from the same DC gain as Eq. (4.2). The coupling matrix

$$C_{k,l} = \kappa_l \sum_{j \in \mathcal{D}_l} g_{k,j} \tag{8.1}$$

is the averaged encoder angle of axis k per radian of feed-forward on axis l; its diagonal is 1 by construction. For the 2025 FEM (Table 8.2) the off-diagonal terms are at most 1.8×10^{-6}, and the per drive gains $g_{k,j}$ across axes are 10^{-5} to 10^{-7} of those within an axis. The largest terms are the elevation and the GIR responses to the azimuth feed-forward, the azimuth having the largest setpoint (−1.70° at 300 s). The static errors left by the uncoupled feed-forward at

Table 8.1: Summary of the verifications.

Item	Verification	Result
Trajectory	regeneration with `atp_trajectory.py` identical to `atp_trajectory.csv`	pass
Tracking	encoders vs setpoint over 300 s (Fig. 1)	pass
Feed-forward	300 s run with/without feed-forward; axes cross-coupling (Table 8.2)	saturation removed; coupling $\leq 1.8 \times 10^{-6}$
Kinematics	compensated RBMs (Fig. 3), segment tip-tilt drift (Fig. 2)	drift removed
GIR rotation	unit tests `zero_angle`, `center_segment`, `no_rotation` and the deformation rotation test	see §8.4
Pointing	unit tests `azimuth_elevation`, `elevation`, `gir`	see §8.4
Optical model	unit tests `pointing_error`, `pointing_rotation_error`	pass
End-to-end	image centroid vs Eq. (7.6) (Figs. 4, 5)	≤ 0.03 mas rms after 2 s
GIR error	GIR tracking error exaggerated by 10^4 (Fig. 6)	fitted gain 0.999

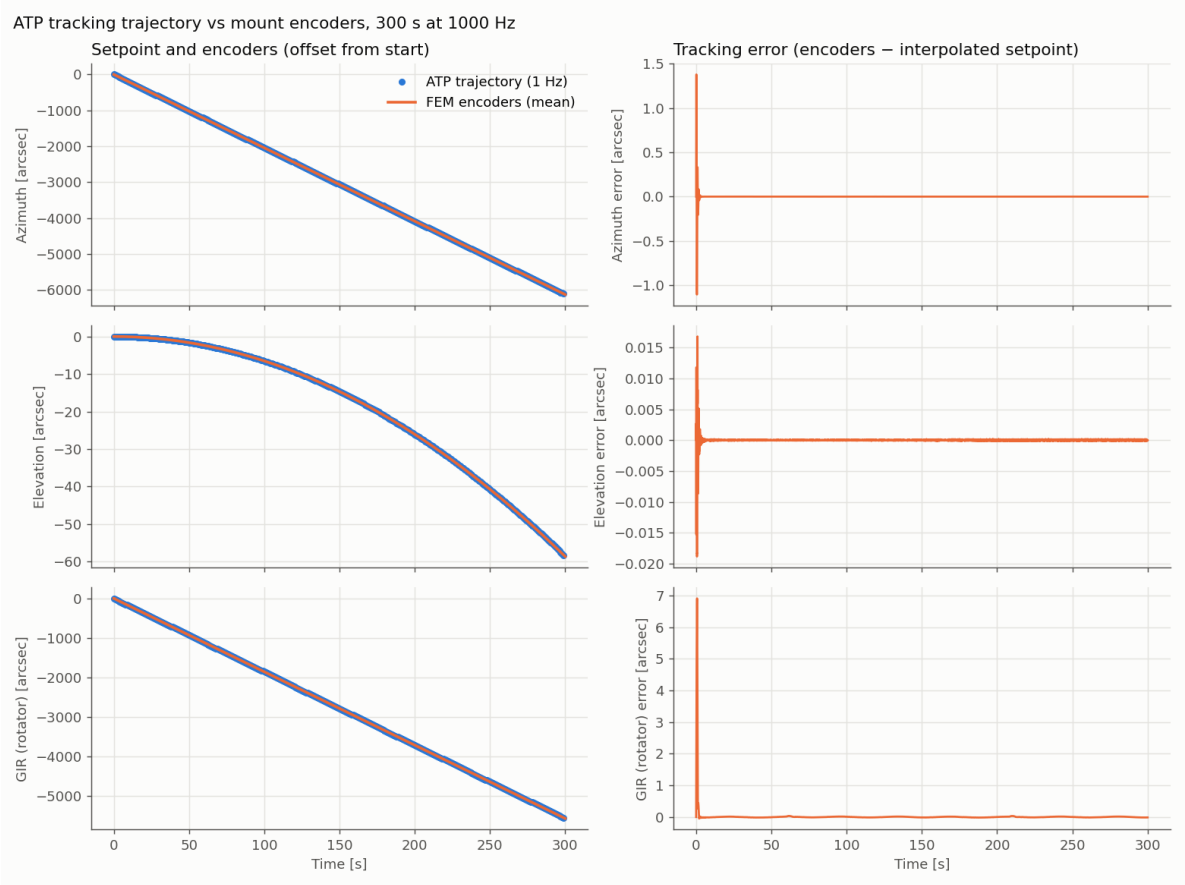


Figure 1: ATP trajectory vs mount encoders (left) and tracking error (right), 300 s at 1 kHz.

the end of the trajectory, $\sum_{l\neq k} C_{k,l}\theta_l$, are 5.0×10^{-8} rad (10 mas) in elevation, 7.3×10^{-9} rad (1.5 mas) for the GIR and below 1 mas in azimuth. They build up slowly (about 34 μas/s in elevation) and are removed by the mount feedback for a negligible torque, about 0.5 N m per elevation drive against the 800 kN m saturation, which is why no residual appears in the tracking error of Figure 1. A decoupled feed-forward, C^{-1}, would change the torques by at most 1.8×10^{-6} of the diagonal terms and is not needed.

Table 8.2: Mount axes cross-coupling $C_{k,l}$ of the feed-forward: averaged encoder angle of axis k (rows) per radian of feed-forward on axis l (columns), 2025 FEM.

	az	el	GIR
az	1	-1.8×10^{-6}	-2.4×10^{-9}
el	-1.7×10^{-6}	1	-3.6×10^{-9}
GIR	-2.5×10^{-7}	4.4×10^{-9}	1

8.3 Kinematic compensation

The kinematic compensation was reached in three steps, each verified on the segment tip-tilt from the LOM, the most sensitive indicator of a residual rigid motion:

1. *Geometric removal* (`scripts/rigid_body.py`): the rigid motion is modeled as the linearized rotation of the segment vertices and frames about the azimuth axis $(0, \sin 30°, \cos 30°)$ and the elevation axis $(-1, 0, 0)$ of the 30° zenith FEM. The mismatch between this geometric model and the FEM kinematics left a segment tip-tilt drift that grows with the azimuth angle.

2. *FEM DC gain* (Eq. 5.2) applied to the M1 and M2 rigid body motions only: the geometric mismatch is gone, but the segments still drift radially by up to 15–20 mas in 50 s (Figure 2, left). The same drift is obtained with a FEM and M1 only model (`trajectory-mount` demo), which pointed to the M1 hardpoint forces: their kinematic component is offloaded by the M1 actuators force loop. A closed-loop calibration of the azimuth column of K was tried as a workaround.

3. *Compensation of all the FEM outputs* but the mount encoders, including the control feedback (Section 5.3): the drift disappears without any calibration. Over 30–300 s the segment tip-tilt stays within ±1.5 mas with no secular trend (Figure 2, right).

With the final compensation, the M1 and M2 rigid body motions show the start-up transient (tens of µm, sub-arcsecond) decaying to zero within about 20 s and no motion following the 1.7° azimuth rotation (Figure 3).

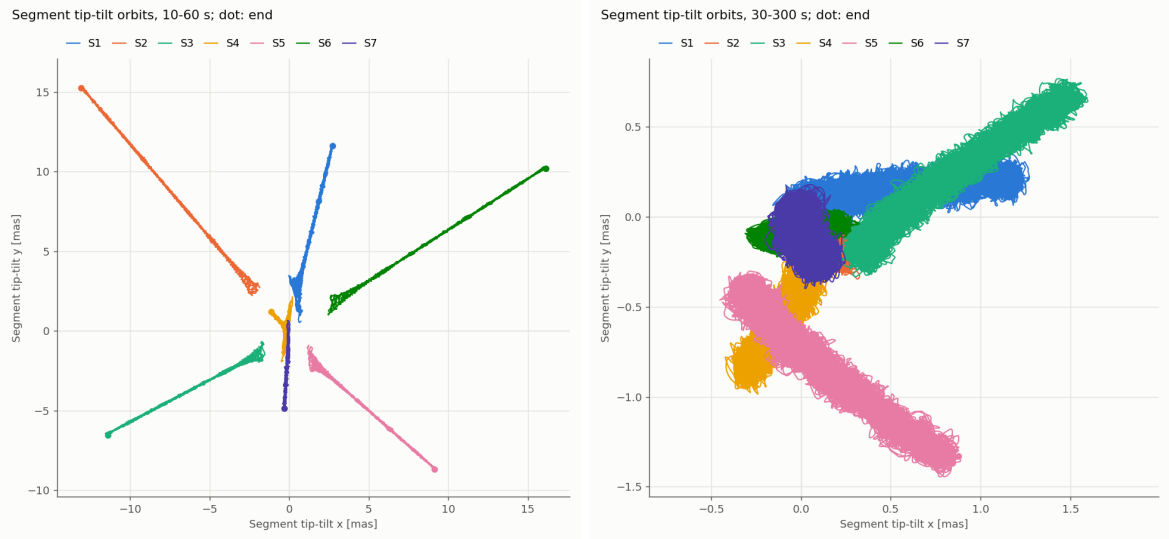


Figure 2: Segment tip-tilt orbits. Left: FEM DC gain kinematics on the M1 and M2 rigid body motions only, 10–60 s. Right: kinematic compensation of all the FEM outputs, 30–300 s.

8.4 Unit tests

`gmt_dos-clients_crseo` (`optical_model/read/pointing.rs`). The tests use a V band camera ($\lambda/(4D)$ sampling binned 2×2) in absolute pointing along the optical axis; results of the re-run:

- `pointing_error`: on-axis source, pointing errors of 20 mas along x then y. The frame centroid moves to $(-19.758, 0.001)$ mas and $(0.003, -19.752)$ mas, i.e. by $-\boldsymbol{p}$ (the centroid of the truncated frame is $\sim$1% short), and the wavefront tip-tilt stays at 0.000 mas. The test passes within 5% $|\boldsymbol{p}| + 0.01$ mas on the image and 0.01 mas on the tip-tilt.

- `pointing_rotation_error`: source 100 mas off-axis along x. Without error the centroid is at $(98.313, -0.009)$ mas; with $\Delta\gamma = 0.04$ rad it moves to $(98.230, -3.964)$ mas, i.e. by $(-0.083, -3.955)$ mas against $((\cos\Delta\gamma - 1)\theta_0, -\sin\Delta\gamma\,\theta_0) = (-0.080, -3.999)$ mas: the image is rotated by $-\Delta\gamma$. Resetting the error restores the initial position.

- `pointing_scan` (ignored by default) scans the pointing error over 1.5 pixel and compares the absolute pointing image motion to the one of an equivalent wavefront tilt. It revealed the CEO imaging axis convention fixed in `crseo-sys` 1.3.3.

`trajectory-servos-mount`.

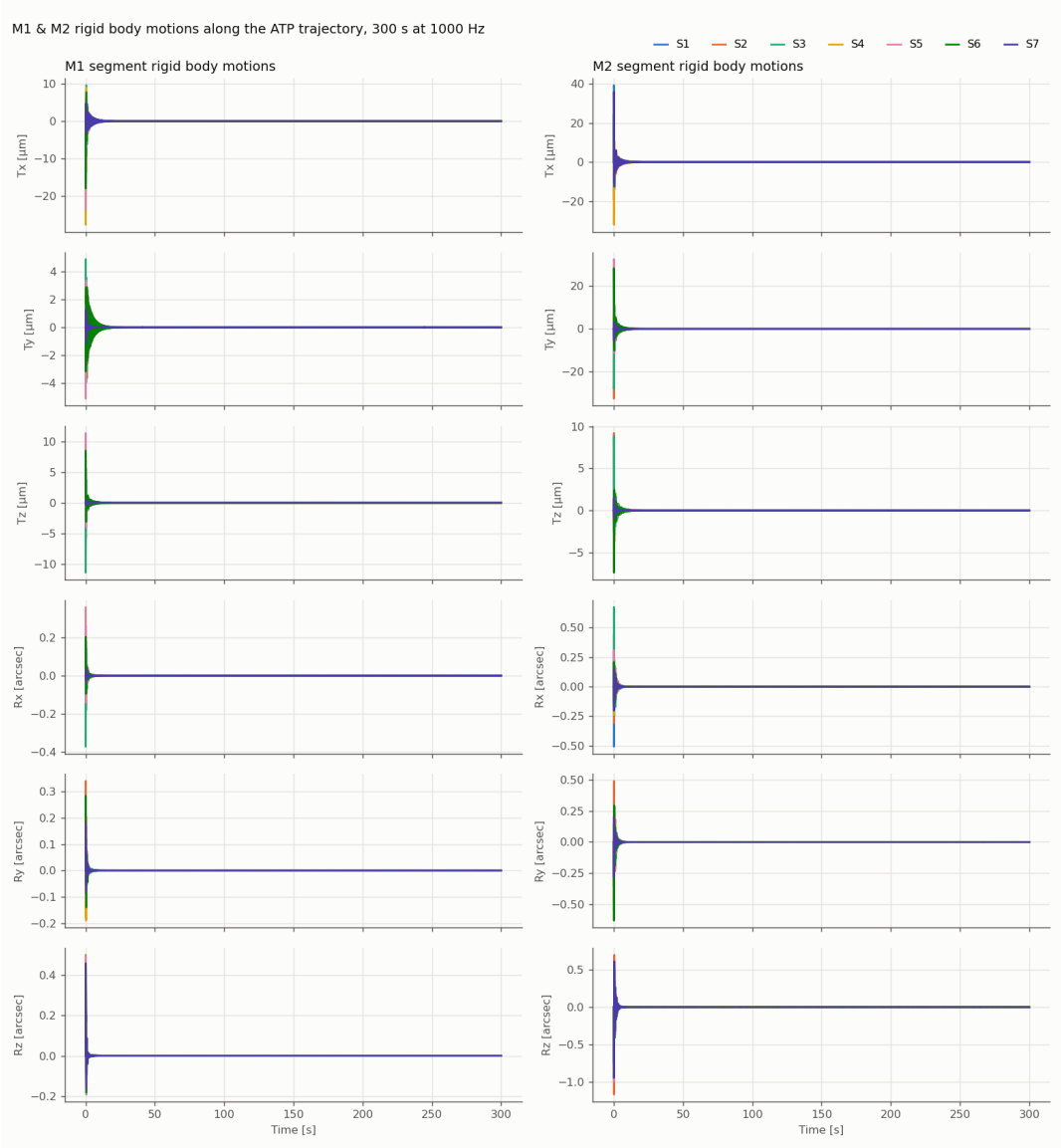


Figure 3: M1 and M2 segment rigid body motions with kinematic compensation along the 300 s trajectory.

- `pointing::tests::azimuth_elevation`: for a trajectory starting at 45° elevation, encoders $(1, 2)$ µrad away from the setpoint give $\boldsymbol{p} = (10^{-6}\cos(45° + 2\,\mu\text{rad}), 2 \times 10^{-6})$; a quarter turn of the GIR maps $x \to y$ and $y \to -x$.

- `pointing::tests::elevation`: for a trajectory starting at 45° elevation with the telescope 15° lower and a 1 µrad azimuth error, $p_x = 10^{-6}\cos 30°$: the cos(el) factor follows the telescope elevation.

- `pointing::tests::gir`: with the GIR tracking a field rotation of 3 µrad and an encoder at 1 µrad, $\Delta\gamma = -2$ µrad; with the GIR at rest, $\Delta\gamma = -q$.

- `gir_rotation::tests::zero_angle`: no rigid body motion for $\gamma = 0$, for M1 and M2.

- `gir_rotation::tests::center_segment`: for $\gamma = 10^{-2}$ rad, M1 S7 (on axis) has no translation and a rotation of exactly γ about z.

- `gir_rotation::tests::no_rotation`: for $\gamma = 0$, any deformation of the M1 and M2 segments is returned unchanged.

- `gir_rotation::tests::deformation_rotates_with_the_mirror`: M1 S7 translated by 1 µm along x and tilted by 2 µrad about x, rotated by $\gamma = 90°$, is translated along y and its normal, initially tilted towards $-y$, is tilted towards $+x$.

8.5 End-to-end verification of the image motion

`scripts/plot_tiptilt_vs_tracking.py` compares the centroid of the image on the camera to the prediction of Eq. (7.6) built from the logged `PointingError`, `PointingRotationError` and `TipTilt`. Frames where the predicted image is outside 80% of the FOV are discarded (the centroid is biased near the frame edges) and a static centroid bias, estimated after 2 s, is removed. The off-axis angle θ_0 of the run is given to the script as its 6th argument (default $0.25''$).

Figure 4 is a 30 s run with GIR tracking and an on-axis source (`OFF_AXIS=0`), so it verifies the pointing error and the structure tip-tilt terms of Eq. (7.6), the rotation term being zero on axis. The static centroid bias is $(0.97, 0.98)$ mas, a quarter of a pixel. During the first second the pointing error reaches about $1''$ in x and the image leaves the field. The image then follows $-\boldsymbol{p}$, and after a few seconds the structure tip-tilt dominates over the pointing error. Re-running the script on the stored data gives the rms values of Table 8.4: the image motion is explained to 0.27 mas rms in x over the whole run, the largest residuals (up to 4 mas) occurring during the first 1.5 s when the image is close to the edge of the field, to 0.03 mas rms after 2 s and to 0.007 mas after 5 s. After 10 s the image motion settles to 0.1–0.3 mas, dominated by the structure tip-tilt, the on-sky tracking error being below 0.1 mas.

Figure 5 is a 30 s run with `GIR_TRACKING=0`: the field rotation term grows linearly and reaches 0.69 mas at 30 s, against 0.676 mas expected from $\theta_0\,|q(30\,\mathrm{s})|$, and the image motion follows it once the start-up transient has decayed.

With GIR tracking, the rotation term is the GIR tracking error times the off-axis angle: at most 3.3×10^{-5} rad$\times0.25''\simeq0.008$ mas during the start-up transient, when the residual reaches several mas, and about 10^{-4} mas afterwards, below the 0.002–0.007 mas residual. It cannot be measured as such. To verify the rotation path in that configuration, a 30 s run with GIR tracking and the source 0.25″ off-axis was made with `PointingRotationError` exaggerated by a gain of 10^4 (`POINTING_ROTATION_GAIN=1e4`, Figure 6): the rotation error peaks at 0.33 rad, moving the image by up to 82 mas, and the GIR tracking ripple after the transient moves it by about 0.1 mas. The image motion y minus $(-\boldsymbol{p}+\boldsymbol{t}_{\mathrm{TT}})$ is fitted by the rotation term times a gain (Table 8.3): the fitted gain is 0.999 after 5 s, and the rotation term reduces the residual from 0.106 mas to 0.0013 mas rms. The only large residual, 50 mas at 0.3 s in x, occurs when the pointing transient brings the off-axis image to the edge of the field. The rotation by the GIR tracking error is therefore applied as predicted, and, the term being linear in $\Delta\gamma$, this holds for the actual, 10^4 times smaller, error.

Table 8.3: Image motion along y explained by the rotation term, 30 s run with GIR tracking, source 0.25″ off-axis and `PointingRotationError` exaggerated by 10^4 (1468 out of 1500 frames in the field). Residual rms [mas] without and with the rotation term, and fitted gain of the term.

	rotation term	residual without	residual with	fitted gain
$t>0\,\mathrm{s}$	5.904	5.802	0.231	0.982
$t>2\,\mathrm{s}$	0.120	0.120	0.010	0.992
$t>5\,\mathrm{s}$	0.106	0.106	0.0013	0.9992

Table 8.4: Image motion and residual rms [mas] of the 30 s on-axis run with GIR tracking (1472 out of 1500 frames in the field).

	image motion		$-\boldsymbol{p}$		residual	
	x	y	x	y	x	y
$t>0\,\mathrm{s}$	28.35	4.24	27.52	1.36	0.274	0.034
$t>2\,\mathrm{s}$	3.35	1.80	2.37	0.21	0.028	0.014
$t>5\,\mathrm{s}$	0.49	0.34	0.11	0.07	0.007	0.002

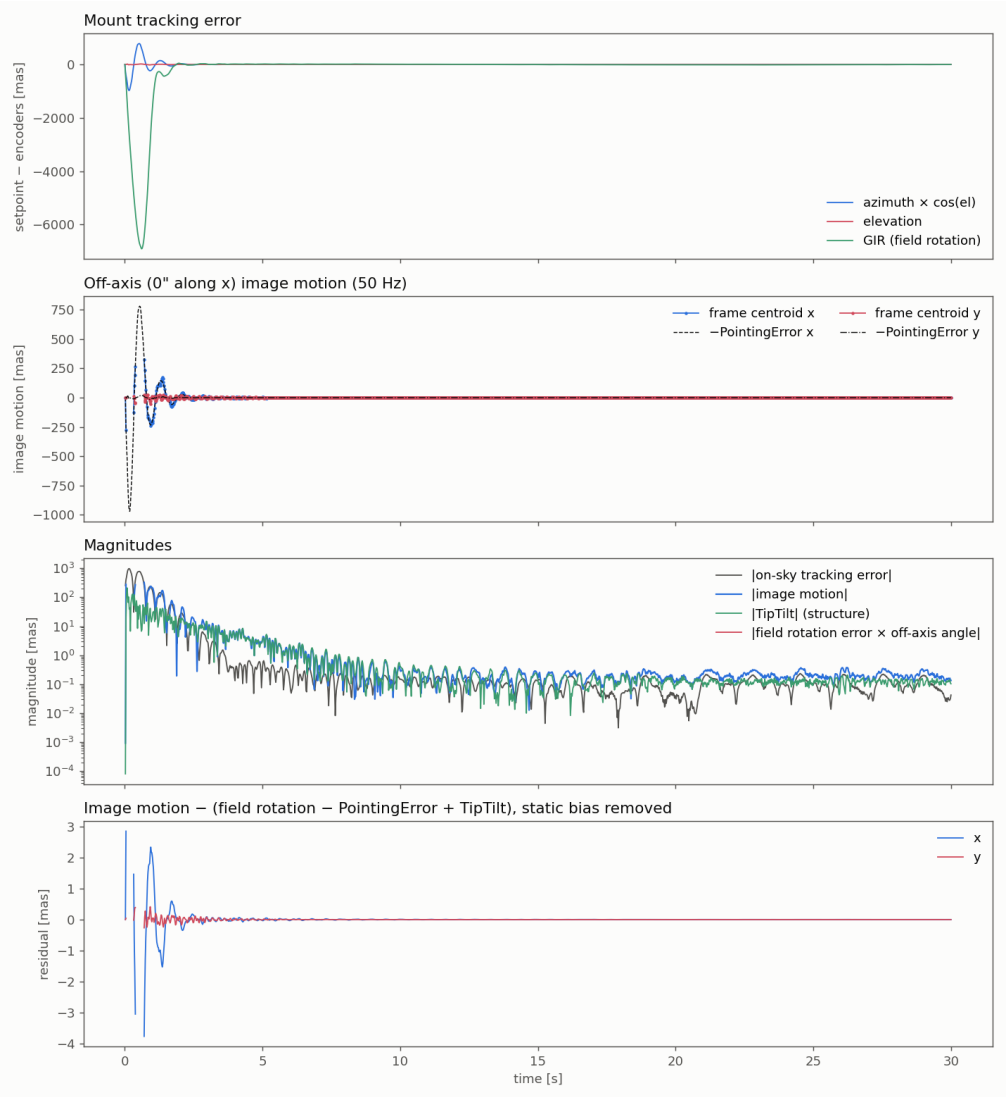


Figure 4: Mount tracking error and image motion, 30 s on-axis run with GIR tracking. From top to bottom: tracking error, image centroid vs $-\boldsymbol{p}$, magnitudes of each term of Eq. (7.6), residual.

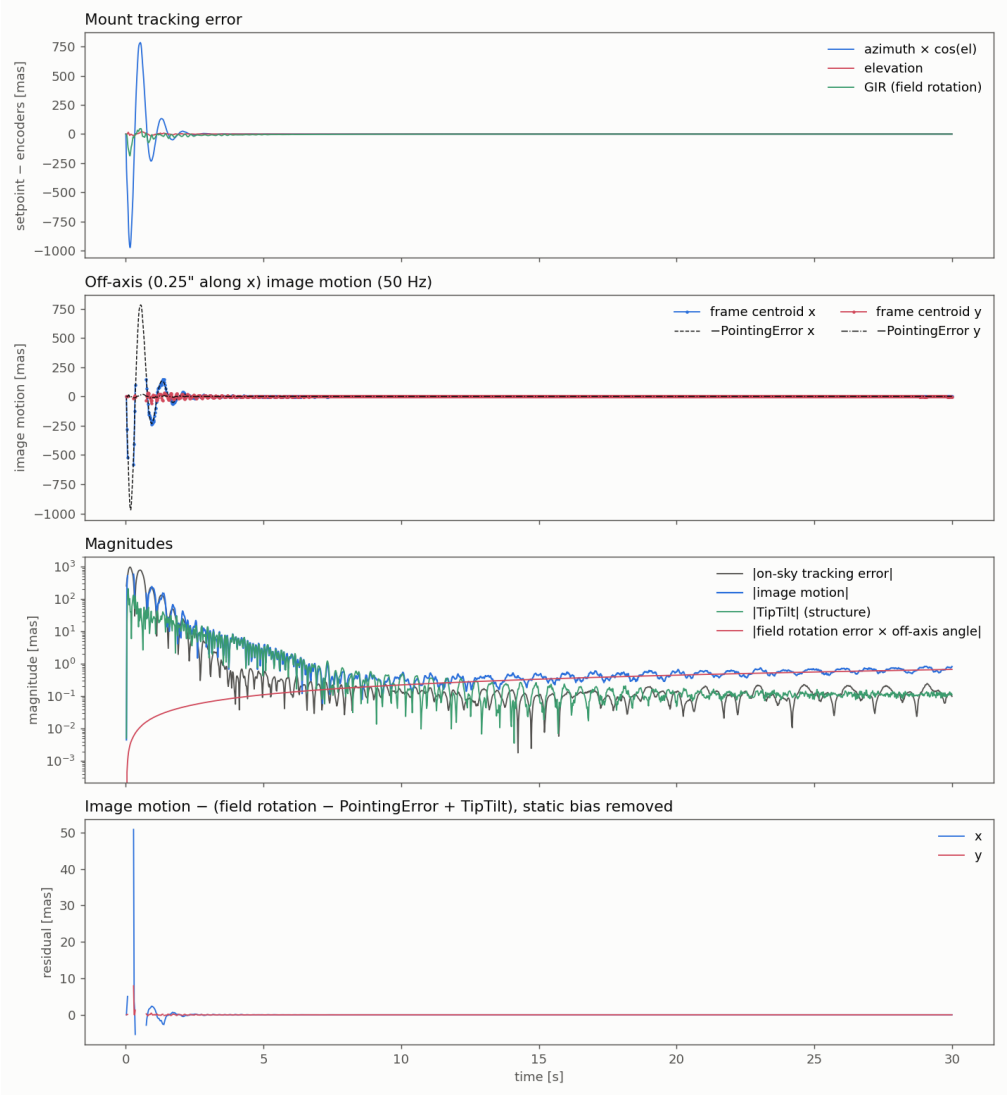


Figure 5: Same as Figure 4 for a 30 s run with `GIR_TRACKING=0` and a source 0.25″ off-axis: the image rotates with the field.

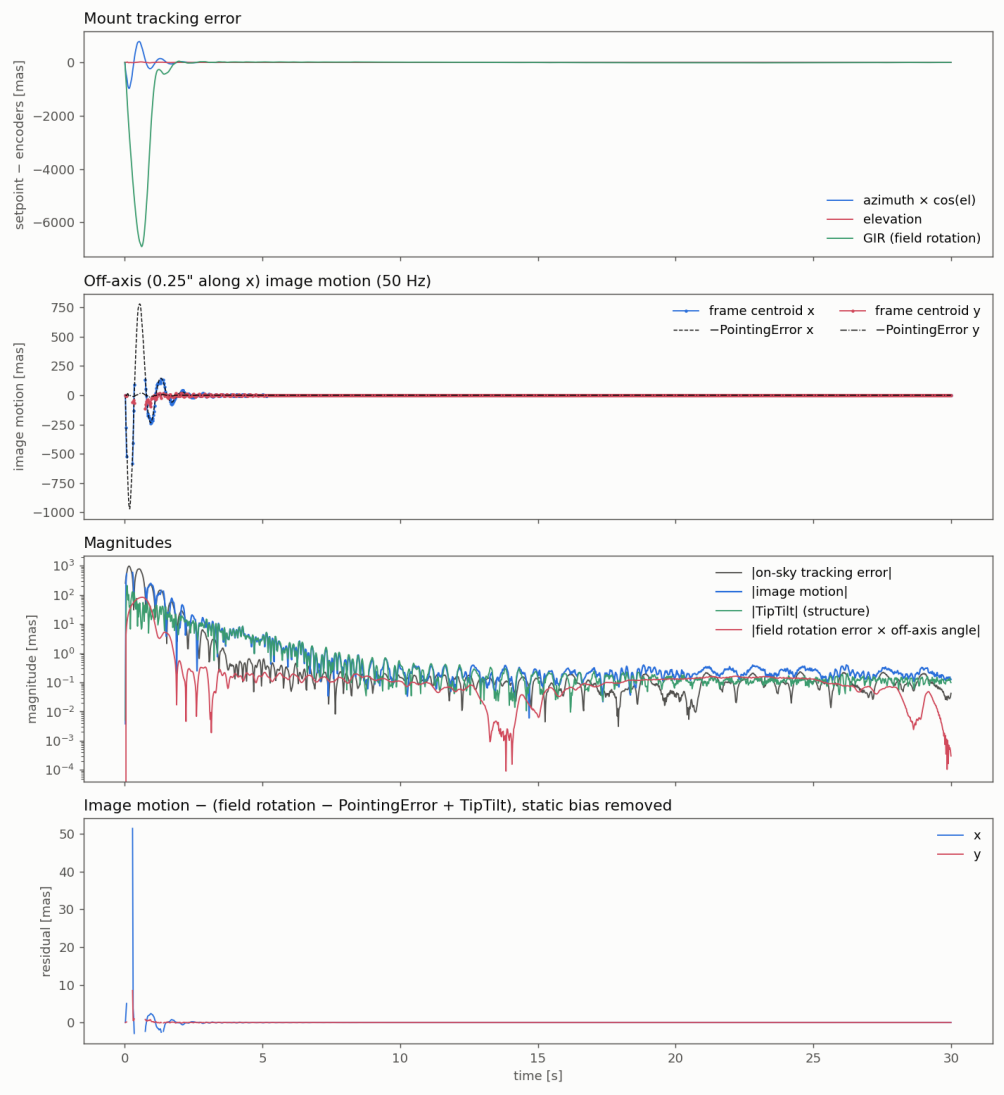


Figure 6: Same as Figure 4 for a 30 s run with GIR tracking, a source 0.25″ off-axis and the GIR tracking error exaggerated by 10^4 (`POINTING_ROTATION_GAIN=1e4`).

9 Computational performance

The run time of the model was measured with the FEM state space model solved on the GPU (`cuda` feature) and without flowchart (`FLOWCHART=none`), for the source 0.25″ off-axis with GIR tracking. The run time is split between the setup, i.e. loading the FEM and computing the kinematic compensation and the feed-forward gains from its DC gain, and the simulation itself, from the launch to the completion of the actors model.

9.1 Full trajectory

Table 9.1 gives the run time of the full 300 s ATP trajectory with the `crseo` optical model on an AWS g7e.4xlarge instance (NVIDIA RTX PRO 6000 Blackwell Server Edition GPU, 16 vCPUs, Intel Xeon Platinum 8559C), the FEM being read from S3 through `mount-s3`. The simulation runs faster than real time, at 0.65 s per simulated second. The 3.9 GB of camera frames are kept in memory and written at the end of the run.

Table 9.1: Run time of the 300 s ATP trajectory with the `crseo` optical model, AWS g7e.4xlarge (RTX PRO 6000 Blackwell).

Simulation (launch to completion of the model)	194.0 s
Setup and data writing	57.4 s
Total	251.4 s
Peak memory	20 GB
Output: 1 kHz data (`model_data_1.parquet`)	18 MB
Output: 50 Hz camera frames (`model_data_20.parquet`)	3.9 GB

9.2 Laptop and cloud GPUs

Table 9.2 compares 30 s runs, two of each, with and without the `crseo` optical model (with the LOM instead) on a laptop (NVIDIA GeForce RTX 3080 Ti Laptop GPU, Intel Core i7-12700H) and on an AWS g7e.12xlarge instance (one of its two RTX PRO 6000 Blackwell GPUs).

Profiling with NVIDIA Nsight Systems shows where the time goes:

- the setup (22 s on the laptop, 31–35 s on AWS with the FEM read from S3) is single-threaded CPU work and does not depend on the GPU;
- each 1 kHz FEM step is dominated by the FP64 matrix-vector product of the modal state space model, which is limited by the GPU memory bandwidth (0.66 ms of the 1.13 ms step on the laptop);

Table 9.2: Run time of 30 s of the trajectory [s], two runs each.

	RTX 3080 Ti Laptop	RTX PRO 6000 Blackwell	speed-up
simulation, LOM	32.4 / 33.0	14.97 / 14.82	2.2
simulation, `crseo`	49.3 / 51.5	19.59 / 19.55	2.6
total, LOM	54.2 / 55.2	45.9 / 45.8	1.2
total, `crseo`	71.5 / 73.6	54.3 / 54.2	1.3

- each 50 Hz step of the `crseo` optical model adds about 12 ms on the laptop and 3 ms on the RTX PRO 6000, mostly the FP64 ray tracing kernels of CEO through M1 and M2 and the truss vignetting, the FEM and CEO sharing the default CUDA stream.

Changes to CEO and to the FEM CUDA solver were tested and not retained: removing the per step device memory allocations and synchronous copies of CEO and pre-selecting the segment in the ray tracing kernel left the results bit-identical but changed the run time by less than 1 %, and running the FEM solver on its own CUDA stream made the run 17–24 % slower, the FEM and CEO kernels then competing for the same GPU and delaying the FEM steps of the 1 kHz loop. Running the FEM and the optical model on two different GPUs was not tested.